



Internship Take-Home: Agentic AI Application Development

First of all, thanks for your interest. This exercise is designed to show us how you think about building agentic AI applications — not just calling a language model, but designing a system that plans, uses tools, handles failure, and knows whether it succeeded. We care far more about your reasoning and engineering judgment than about a polished, feature-complete product. Please read the whole document before you start.

The Challenge

Build a small agentic "research assistant" application.

Given a natural-language goal (for example, "*Compare the population growth of Florida and Texas over the last decade and tell me which grew faster in percentage terms*"), your agent should plan an approach, use tools to gather and process information, and return a synthesized, cited answer — correcting itself when a step fails or returns something unexpected.

The domain doesn't matter much; pick something you can get real data for. What matters is that the task genuinely requires multiple steps and tools, so a single prompt-to-model call isn't enough.

Core Requirements (everyone should aim for these)

1. **A real agent loop, not a single call.** The agent should follow some form of *plan* → *act* → *observe* → *decide-what's-next* cycle, and be able to take multiple steps before answering.
2. **At least two or three tools.** For example: web search or a provided document set, plus a calculator / code-execution tool, plus anything else useful. At least one tool should be able to fail or return junk (e.g. a search that finds nothing), so we can see how you handle it.



3. **Error handling and recovery.** Retries, fallbacks, or re-planning when a tool errors, times out, or returns something unusable.
 4. **State across steps.** The agent should carry context/memory from one step to the next rather than treating each step in isolation.
 5. **Structured, grounded output.** Return a clear final answer with citations or references to where the information came from.
 6. **A minimal evaluation.** Provide 5–10 test inputs, define what "correct" means for them, and show how your agent did. This can be as simple as a script that runs the cases and prints pass/fail with your reasoning.
 7. **A README covering:** how to run it, your key design decisions and tradeoffs, known limitations, and what you'd do with more time.
-

Stretch Goals (optional — pick what interests you)

You don't need to do all of these. Choose the ones that let you show your strengths. Depth on one is better than shallow attempts at many.

- **Self-correction / reflection:** a critique step where the agent reviews its own draft answer and revises it.
 - **Ambiguity handling:** detect when a request is under-specified and ask a clarifying question instead of guessing.
 - **Guardrails:** reduce hallucination — e.g. refuse to answer when evidence is insufficient, and say so.
 - **Cost & latency awareness:** track tokens/calls/time, and show one optimization you made and why.
 - **Evaluation harness:** a more rigorous eval with metrics (accuracy, tool-call success rate, etc.) rather than eyeballing.
 - **Architecture:** a planner/executor split, a multi-agent design, or a clearly reasoned state-graph — with a short justification for the choice.
-

Deliverables

1. A Git repository (GitHub link or a zip) with your code.
2. A README as described above.



3. A short design note (½–1 page is plenty, or a 3–5 minute screen-recorded walkthrough if you prefer talking over writing) explaining the *why* behind your architecture and the most interesting problem you hit.
 4. Your evaluation cases and results.
-

Ground Rules

- **Time-box:** aim for ~4–6 hours. Please don't go far beyond this — we'd rather see a smaller thing done well, with honest notes on what you left out, than a sprawling unfinished project. Tell us in the README where you chose to stop and why.
 - **Any language; Python preferred.** Any model provider or framework is fine (LangGraph, LlamaIndex, plain API calls — your choice). Be ready to explain why you picked it.
 - **AI coding assistants are allowed and expected.** We use them too. What we're evaluating is your judgment in directing them and reviewing their output — so please note briefly how you used them.
 - **Keep costs low.** A small/cheap model is completely fine; we're not judging model choice by price.
 - **Ask questions.** If something is ambiguous, a good clarifying email is a positive signal, not a negative one.
-

How We'll Evaluate

Area	What we're looking for	Weight
Agentic design	Sensible loop, tool orchestration, state handling, sound architecture	30%
AI/LLM judgment	Prompt design, grounding, failure handling, thoughtful evaluation	25%
Engineering quality	Readable, structured, reasonably tested code; clean interfaces	20%
Communication	Clear README and design note; honest about tradeoffs and limits	15%



Depth / initiative	A stretch goal or insight that goes beyond the minimum	10%
--------------------	--	-----

A good submission typically nails the core requirements with clean code and clear reasoning. A strong submission usually also shows depth on one or more stretch goals.

Follow-Up (for the interview)

Come prepared to discuss:

- Walk us through your agent's control flow. Where could it get stuck in a loop or fail silently?
 - How do you *know* your agent works? How would you evaluate it at scale?
 - Where does it hallucinate or break, and how would you harden it for production?
 - If this needed to handle 10,000 requests a day reliably and cheaply, what would you change?
 - What did you cut for time, and what would you build next?
-

Good luck — we're looking forward to seeing how you think.